

useState / src/step10.jsx

- 원본 파일: `src/step10.jsx`
- 코드 형식: `jsx`

```
import { useState } from 'react'

const INITIAL_TASKS = [
  { id: 'state-basic', title: 'useState 기본 문법 복습', done: true },
  { id: 'immutable-array', title: '배열 불변 업데이트 연습', done: false },
  { id: 'updater-function', title: '함수형 updater 설명하기', done: false },
]

function App() {
  const [tasks, setTasks] = useState(INITIAL_TASKS)
  const [draft, setDraft] = useState('')
  const [filter, setFilter] = useState('all')
  const [editingId, setEditingId] = useState(null)
  const [editText, setEditText] = useState('')

  // 기존 state로 계산할 수 있는 값은 별도 state로 중복 저장하지 않습니다.
  const completedCount = tasks.filter((task) => task.done).length
  const remainingCount = tasks.length - completedCount
  const visibleTasks = tasks.filter((task) => {
    if (filter === 'active') {
      return !task.done
    }

    if (filter === 'completed') {
      return task.done
    }

    return true
  })

  function addTask(event) {
    event.preventDefault()
    const title = draft.trim()

    if (!title) {
      return
    }

    setTasks((previousTasks) => [
      ...previousTasks,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
      },
    ])
    setDraft('')
  }

  function toggleTask(taskId) {
    setTasks((previousTasks) =>
      previousTasks.map((task) =>
        task.id === taskId ? { ...task, done: !task.done } : task,
      ),
    )
  }

  function startEditing(task) {
    setEditingId(task.id)
    setEditText(task.title)
  }

  function cancelEditing() {
    setEditingId(null)
    setEditText('')
  }

  function saveTask(event, taskId) {
    event.preventDefault()
  }
}
```

```

const title = editText.trim()

if (!title) {
  return
}

setTasks((previousTasks) =>
  previousTasks.map((task) =>
    task.id === taskId ? { ...task, title } : task,
  ),
)
cancelEditing()
}

function deleteTask(taskId) {
  setTasks((previousTasks) =>
    previousTasks.filter((task) => task.id !== taskId),
  )

  if (editingId === taskId) {
    cancelEditing()
  }
}

function clearCompleted() {
  setTasks((previousTasks) =>
    previousTasks.filter((task) => !task.done),
  )
}

function resetAll() {
  setTasks(INITIAL_TASKS)
  setDraft('')
  setFilter('all')
  cancelEditing()
}

return (
  <main>
    <header>
      <p>STEP 10 · USESTATE PRACTICE</p>
      <h1>학습 할 일 관리자</h1>
      <p>
        문자열, 배열, boolean 속성, 편집 대상과 필터 상태를 useState만으로
        조합하는 종합 실습입니다.
      </p>
    </header>

    <section>
      <form onSubmit={addTask}>
        <label htmlFor="new-task">새 학습 항목</label>
        <div>
          <input
            id="new-task"
            value={draft}
            placeholder="학습할 내용을 입력하세요"
            onChange={(event) => setDraft(event.target.value)}
          />
          <button type="submit" disabled={!draft.trim()}>
            추가
          </button>
        </div>
      </form>

      <div>
        <div>
          <span>전체</span>
          <strong>{tasks.length}</strong>
        </div>
        <div>
          <span>남음</span>
          <strong>{remainingCount}</strong>
        </div>
        <div>
          <span>완료</span>
          <strong>{completedCount}</strong>
        </div>
      </div>
    </section>
  </main>
)

```

```

</div>
<div aria-label="목록 필터">
  {[
    ['all', '전체'],
    ['active', '진행 중'],
    ['completed', '완료'],
  ].map(([value, label]) => (
    <button
      key={value}
      type="button"

      aria-pressed={filter === value}
      onClick={() => setFilter(value)}
    >
      {label}
    </button>
  ))}
</div>

{visibleTasks.length > 0 ? (
  <ul>
    {visibleTasks.map((task) => (
      <li key={task.id}>
        {editingId === task.id ? (
          <form

            onSubmit={(event) => saveTask(event, task.id)}
          >
            <input
              aria-label="할 일 수정"
              value={editText}
              autoFocus
              onChange={(event) => setEditText(event.target.value)}
            />
            <button type="submit" disabled={!editText.trim()}>
              저장
            </button>
            <button
              type="button"

              onClick={cancelEditing}
            >
              취소
            </button>
          </form>
        ) : (
          <div>
            <label>
              <input
                type="checkbox"
                checked={task.done}
                onChange={() => toggleTask(task.id)}
              />
              <span>{task.title}</span>
            </label>
            <div>
              <button
                type="button"

                onClick={() => startEditing(task)}
              >
                수정
              </button>
              <button
                type="button"

                onClick={() => deleteTask(task.id)}
              >
                삭제
              </button>
            </div>
          </div>
        )}
      </li>
    ))}
  </ul>
) : (
  <div>
    <p>할 일을 추가하세요</p>
    <div>
      <input type="text" value={newTaskTitle} />
      <button type="button" onClick={() => addTask()}>
        추가
      </button>
    </div>
  </div>
)}
</div>

```

```

    ) : (
      <p>이 조건에 해당하는 항목이 없습니다.</p>
    )}

    <div>
      <button
        type="button"

        disabled={completedCount === 0}
        onClick={clearCompleted}
      >
        완료 항목 삭제
      </button>
      <button type="button" onClick={resetAll}>
        전체 초기화
      </button>
    </div>
  </section>

  <aside>
    <h2>핵심 내용</h2>
    <p>
      완료 개수와 화면에 보일 목록은 기존 상태로 계산할 수 있으므로 별도
      state로 저장하지 않습니다.
    </p>
  </aside>
</main>
)
}

export default App

```